

Virtual Memory

Operating Systems – EDA093/DIT401

Vincenzo Gulisano
vincenzo.gulisano@chalmers.se



CHALMERS
UNIVERSITY OF TECHNOLOGY



UNIVERSITY OF
GOTHENBURG

Reading instructions

- Chapter 3.3 to 3.6

(optional additional reading: 9.1-9.4.5, 9.5 to 9.7.1 from Silberschatz
Operating System Concepts)

Objectives

- To describe the benefits of a virtual memory system
- To explain the concepts of demand paging, page-replacement algorithms, and allocation of page frames
- To discuss the principle of the working-set model

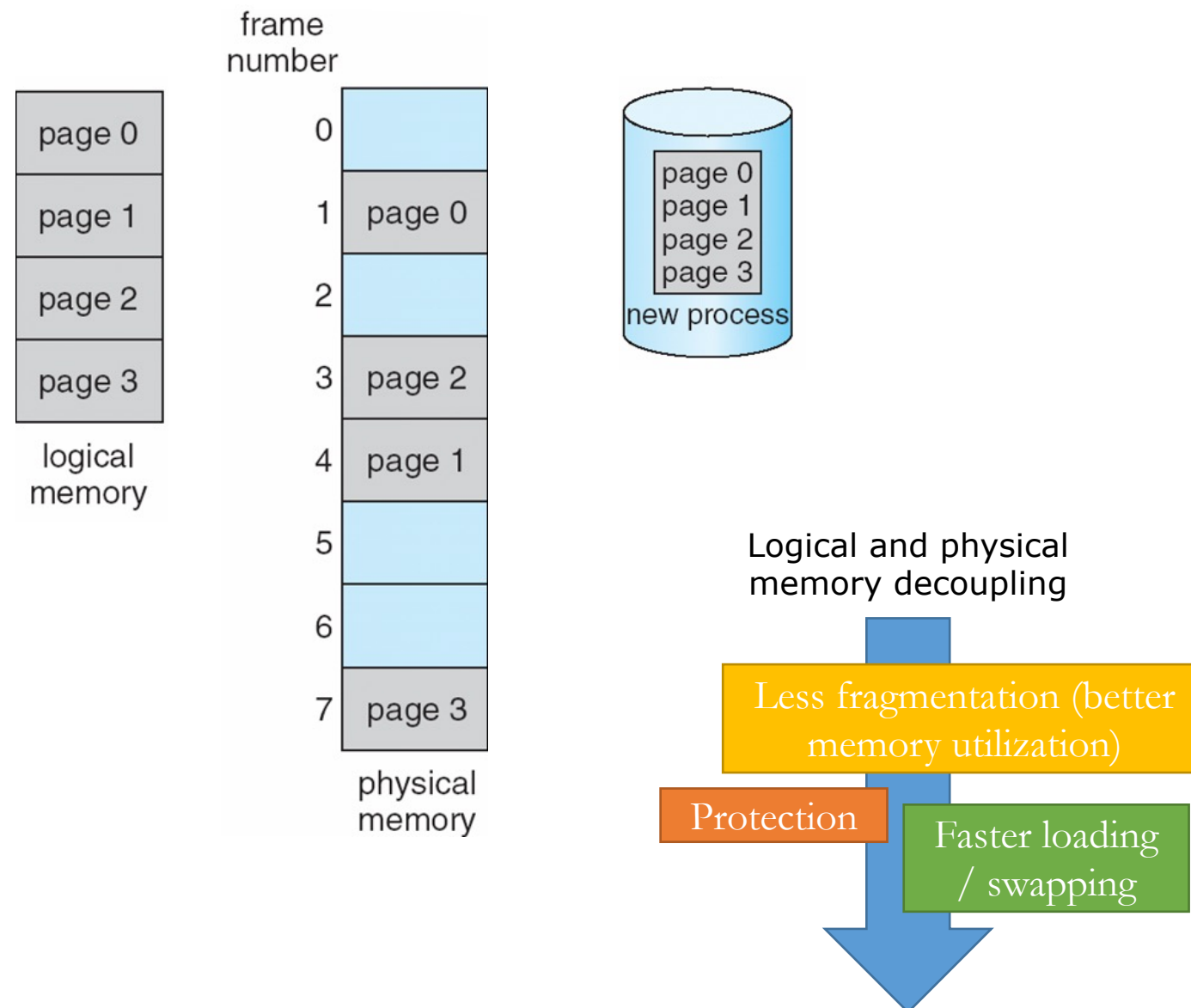
Agenda

- Recap / Introduction
- Demand Paging
- Copy-on-Write
- Page Replacement
- Allocation of Frames
- Thrashing
- Memory-mapped files [Self-reading]

Agenda

- Recap / Introduction
- Demand Paging
- Copy-on-Write
- Page Replacement
- Allocation of Frames
- Thrashing
- Memory-mapped files [Self-reading]

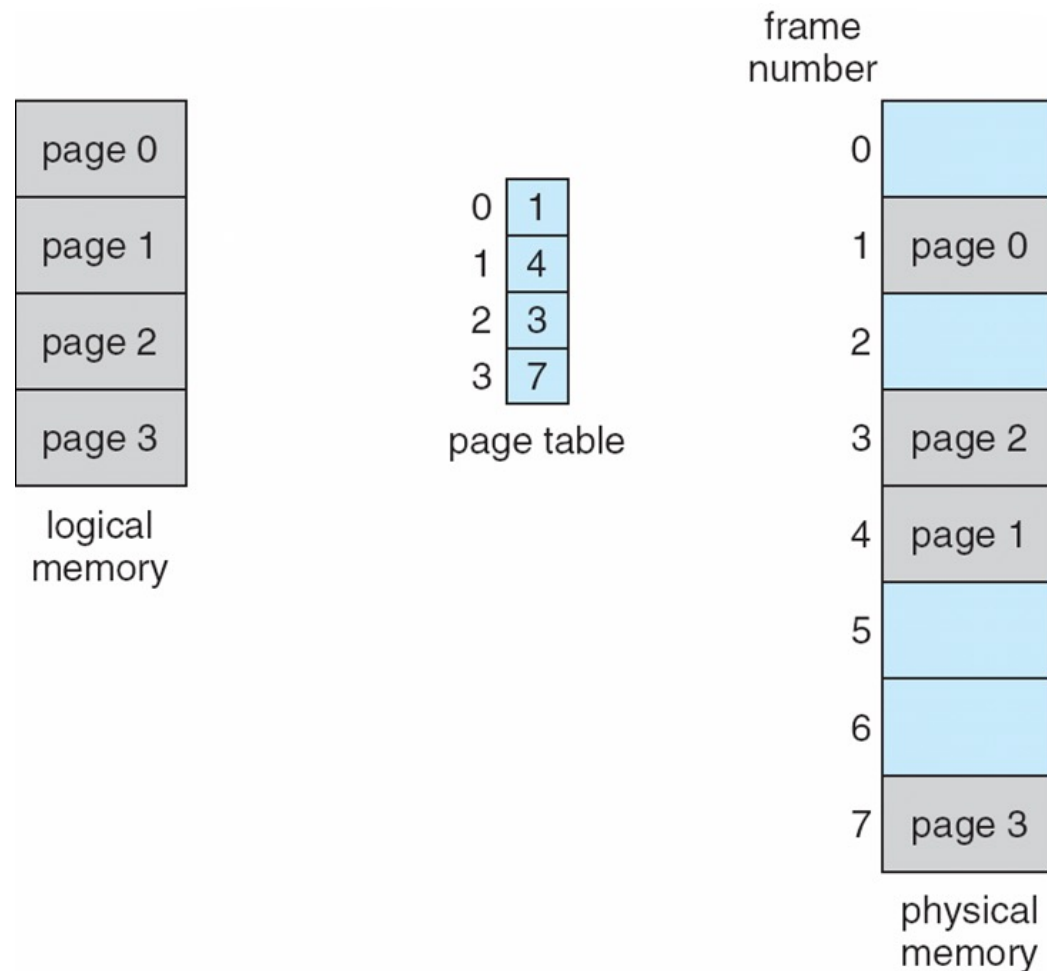
... we discussed paging!



- Divide the logical address space into fixed-size pages
- Divide physical memory into equal-size frames.

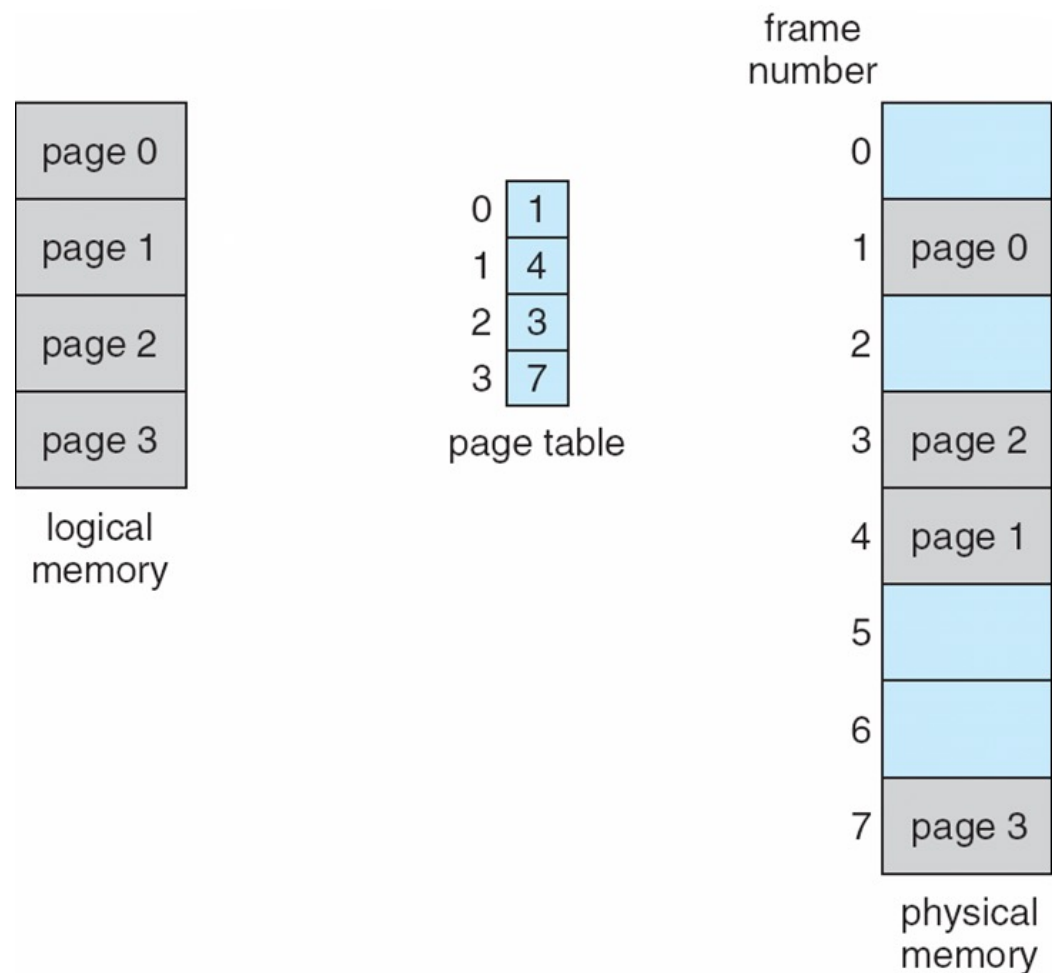
Page size = Frame size!

The intuition behind Virtual Memory



*What would happen if the OS does not load page 3 in frame 7?
(does not load at all, not in a different frame)*

The intuition behind Virtual Memory

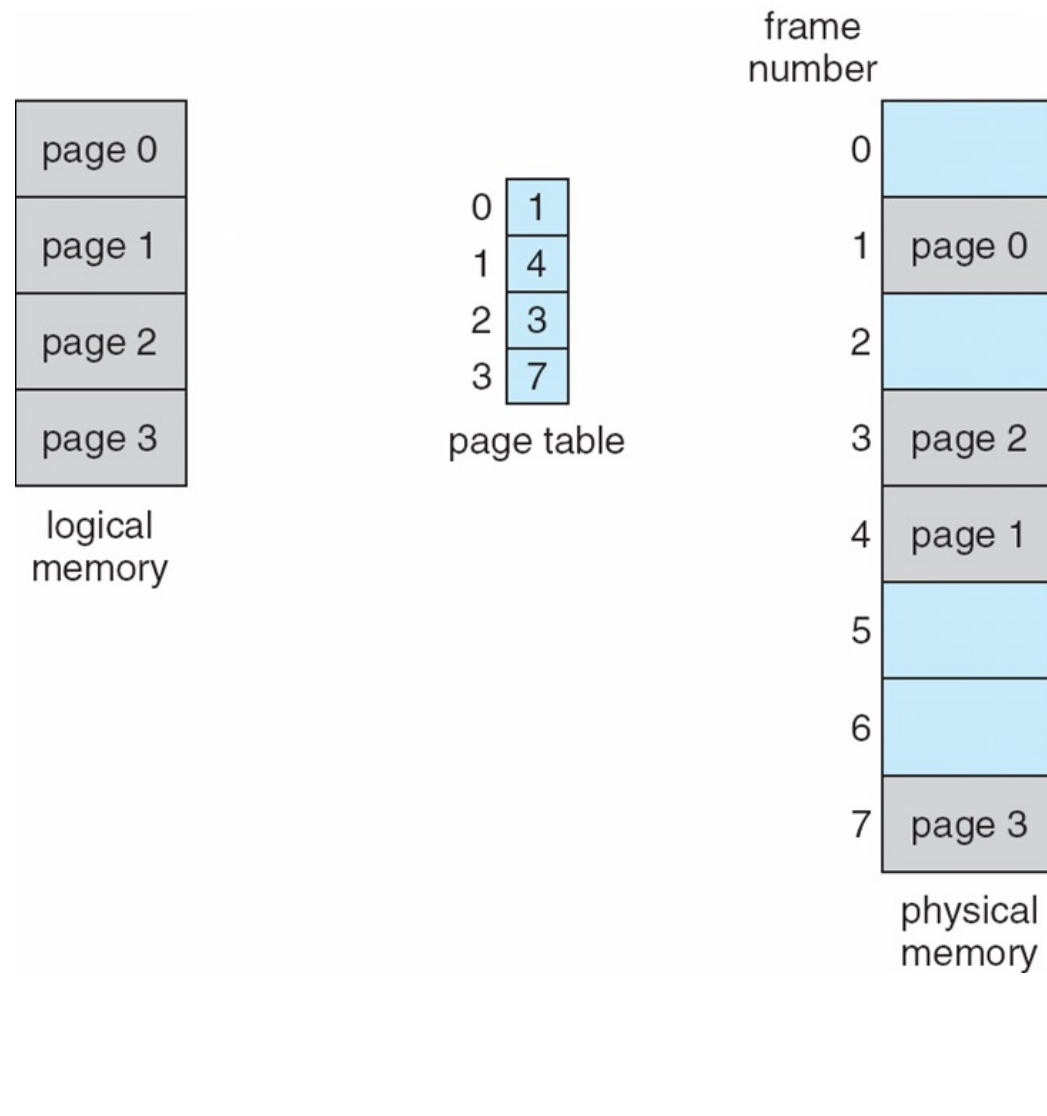


*What would happen if the OS does not load page 3 in frame 7?
(does not load at all, not in a different frame)*



When would that be a problem?

The intuition behind Virtual Memory



*What would happen if the OS does not load page 3 in frame 7?
(does not load at all, not in a different frame)*

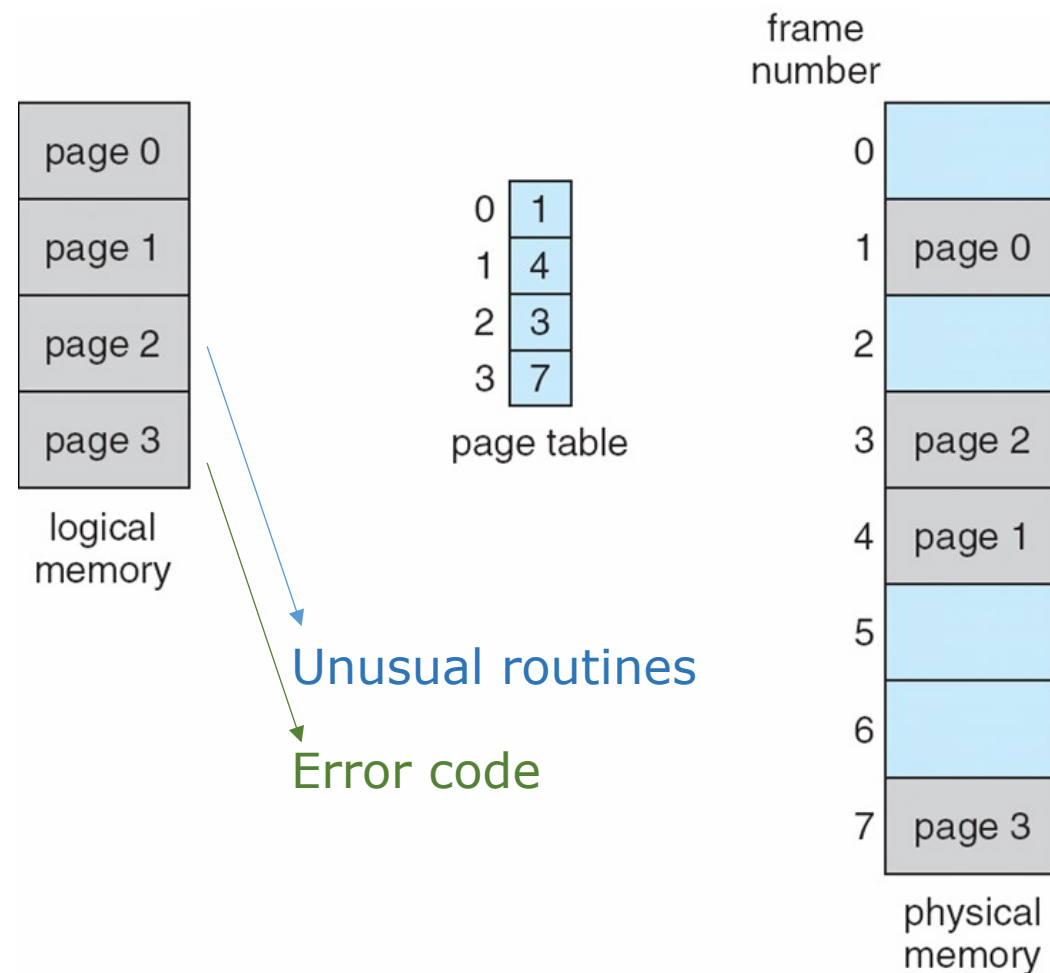


When would that be a problem?



Let's wait until the process tries to access page 3... If it does, then we will load it into frame 7 (or another one, that will be decided later on)!

Virtual Memory in a Nutshell (i)



- Code needs to be in memory to execute, but a process rarely needs all its code and data at the same time.
- If we manage to execute partially-loaded programs:
 - Program no longer constrained by limits of physical memory
 - Each program takes less memory while running -> more programs run at the same time
 - Less I/O needed to load or swap programs into memory -> programs may start sooner because fewer pages are loaded initially (but later page faults add overhead!)

Virtual Memory in a Nutshell (ii)

- What are the costs / trade-offs / complications when executing partially-loaded programs:



- Need a mechanism to check if a page is in the frame or not (track page residency)
- Because of this mechanism, sometimes accessing a page might require more time than expected
- Servicing page faults
- Choosing victim pages
- Transferring pages to or from backing storage

Agenda

- Recap / Introduction
- Demand Paging
- Copy-on-Write
- Page Replacement
- Allocation of Frames
- Thrashing
- Memory-mapped files [Self-reading]

Demand Paging

- Could bring entire process into memory at load time
- Or bring a page into memory only when it is needed
 - Less I/O needed
 - Less resident memory per process can allow more processes to remain active
- Page is needed $\Rightarrow$ reference to it
 - not-in-memory $\Rightarrow$ bring to memory
- **Lazy swapper / pager** – never swaps a page into memory until the process first references it

Basic Concepts

- Pager brings in only those pages into memory
- Need new MMU functionality to implement demand paging (transparently to the application)
- If pages needed are already memory resident
 - No difference from non-demand paging
- If page needed and not memory resident
 - Need to detect and load the page into memory from storage
 - Without changing program behavior
 - Without programmer needing to change code

Valid-Invalid Bit

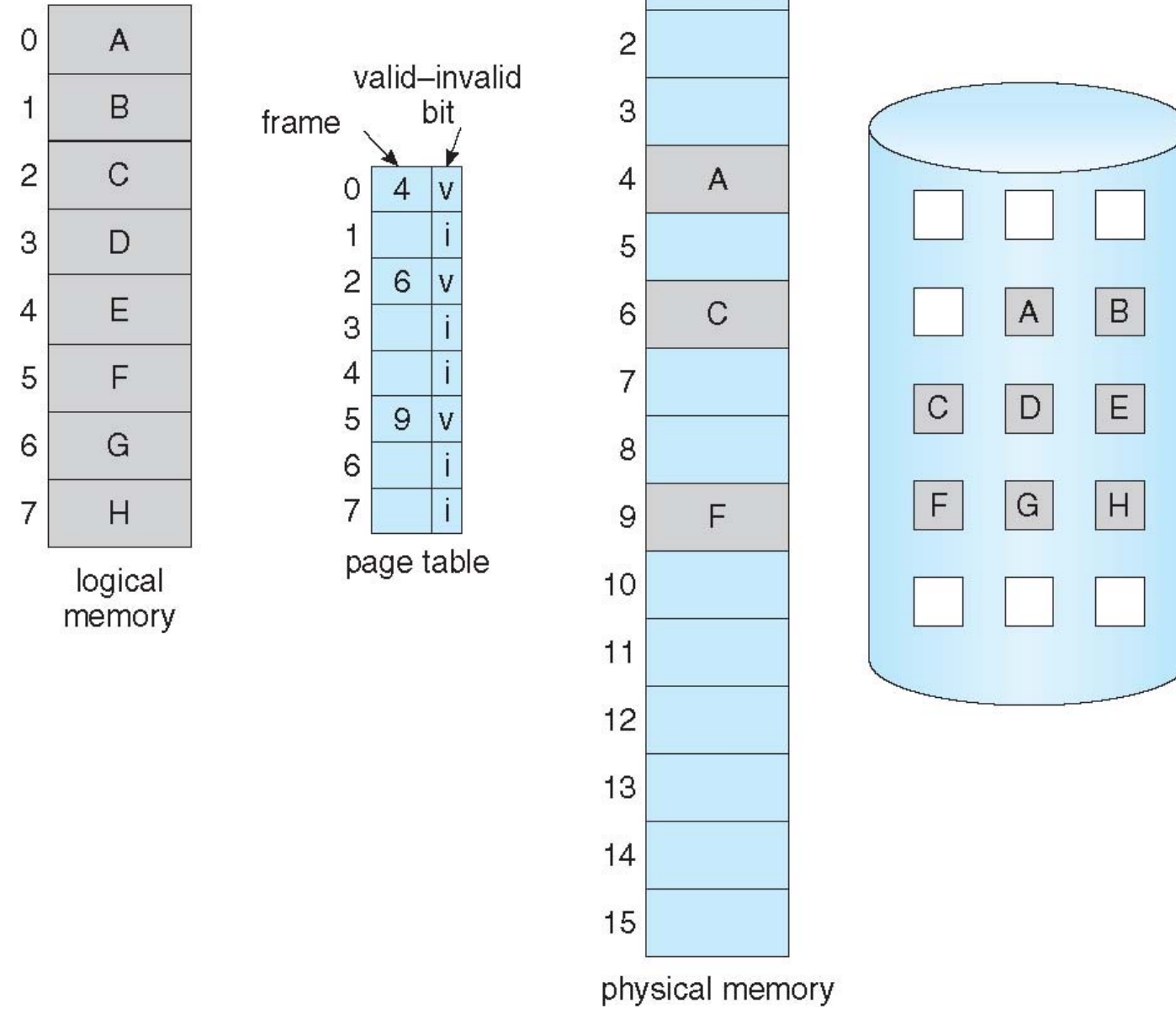
- With each page table entry, a valid–invalid bit is associated
(**v** $\Rightarrow$ in-memory – **memory resident**, **i** $\Rightarrow$ not-in-memory)
- Initially valid–invalid bit is set to i on all entries
- Example of a page table snapshot:

Frame #	valid-invalid bit
	v
	v
	v
	i
...	
	i
	i

page table

- During MMU address translation, if valid–invalid bit in page table entry is **i** $\Rightarrow$ page fault

Page Table When Some Pages Are Not in Main Memory



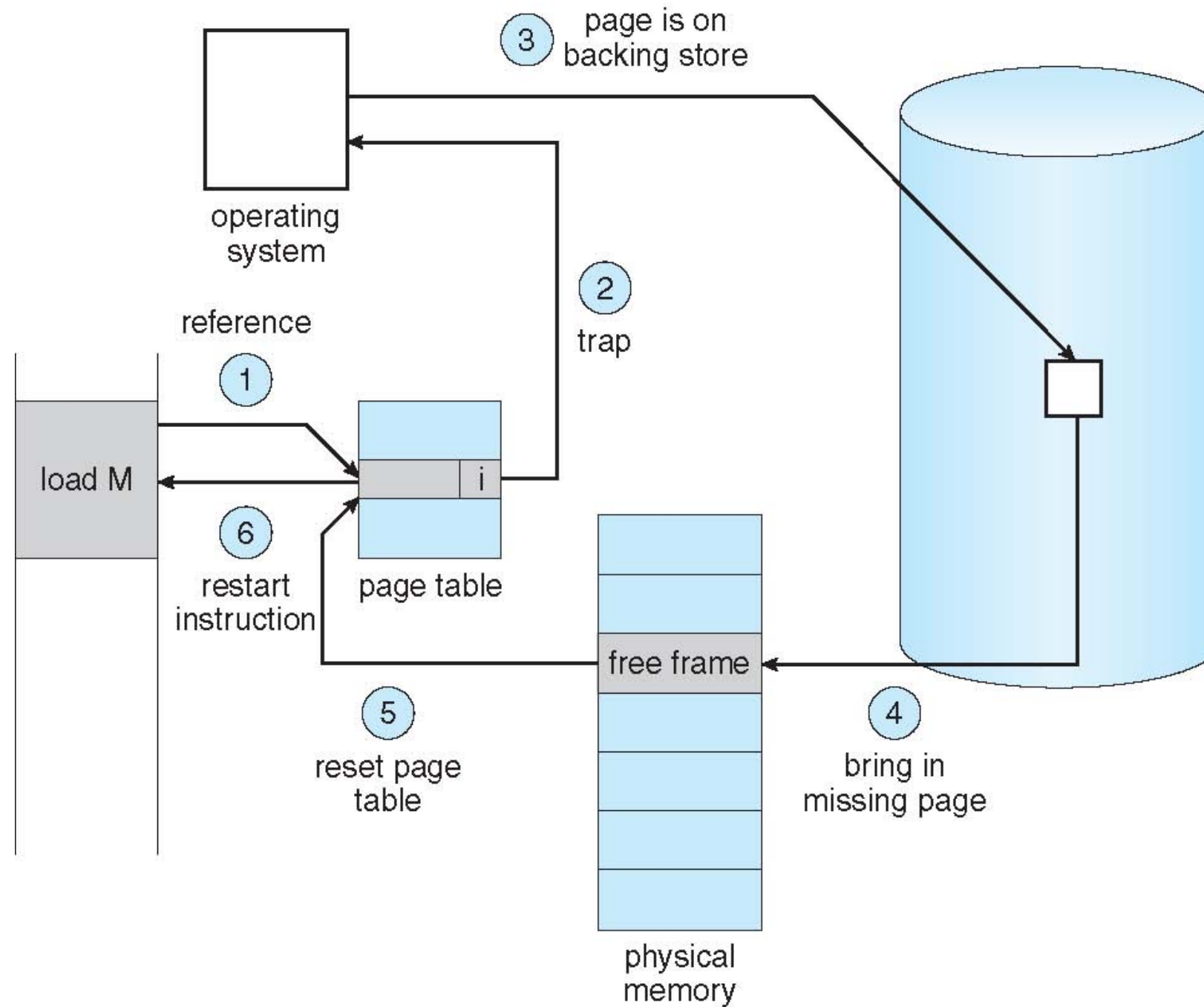
Page Fault

- If there is a reference to a page, first reference to that page will trap to operating system if the page is not resident:

page fault

1. Operating system looks at another table to decide:
 - Invalid reference $\Rightarrow$ abort
 - Just not in memory
2. Find free frame (or select a victim one)
3. Swap page into frame via scheduled disk operation
 1. Might need to swap out what is in the frame if it was not free
4. Reset tables to indicate page now in memory
Set valid bit = **v**
5. Restart the instruction that caused the page fault

Steps in Handling a Page Fault



Aspects of Demand Paging

- **Pure demand paging** – start process with *no* pages in memory
 - OS sets instruction pointer to first instruction of process, non-memory-resident -> page fault
 - The same happens for every other process page on first access
- Hardware support needed for demand paging
 - Page table with valid / invalid bit
 - Secondary memory (swap device with **swap space**)
 - Instruction restart

Performance of Demand Paging

- Stages in Demand Paging (worst case)

1. Trap to the operating system
2. Save the user registers and process state
3. Determine that the trap was a page fault
4. Check that the page reference was legal and determine the location of the page on the disk
5. Issue a read from the disk to a free frame:
 1. Wait in a queue for this device until the read request is serviced
 2. Begin the transfer of the page to a free frame
6. While waiting, allocate the CPU to some other process / thread / user
7. Receive an interrupt from the disk I/O subsystem (I/O completed)
8. Save the registers and process state for the other user
9. Determine that the interrupt was from the disk
10. Correct the page table and other tables to show page is now in memory
11. Wait for the CPU to be allocated to this process again
12. Restore the user registers, process state, and then resume the interrupted instruction

Performance of Demand Paging (Cont.)

- Three major activities
 - Service the interrupt – careful coding means just several hundred instructions needed
 - Read the page – lots of time
 - Restart the process – again just a small amount of time
- Page Fault Rate $0 \leq p \leq 1$
 - if $p = 0$ no page faults
 - if $p = 1$, every reference is a fault
- Effective Access Time (EAT)

$$\begin{aligned} \text{EAT} = & (1 - p) \times \text{memory access} \\ & + p (\text{page fault overhead} \\ & \quad + \text{swap page out [potentially 0]} \\ & \quad + \text{swap page in }) \end{aligned}$$

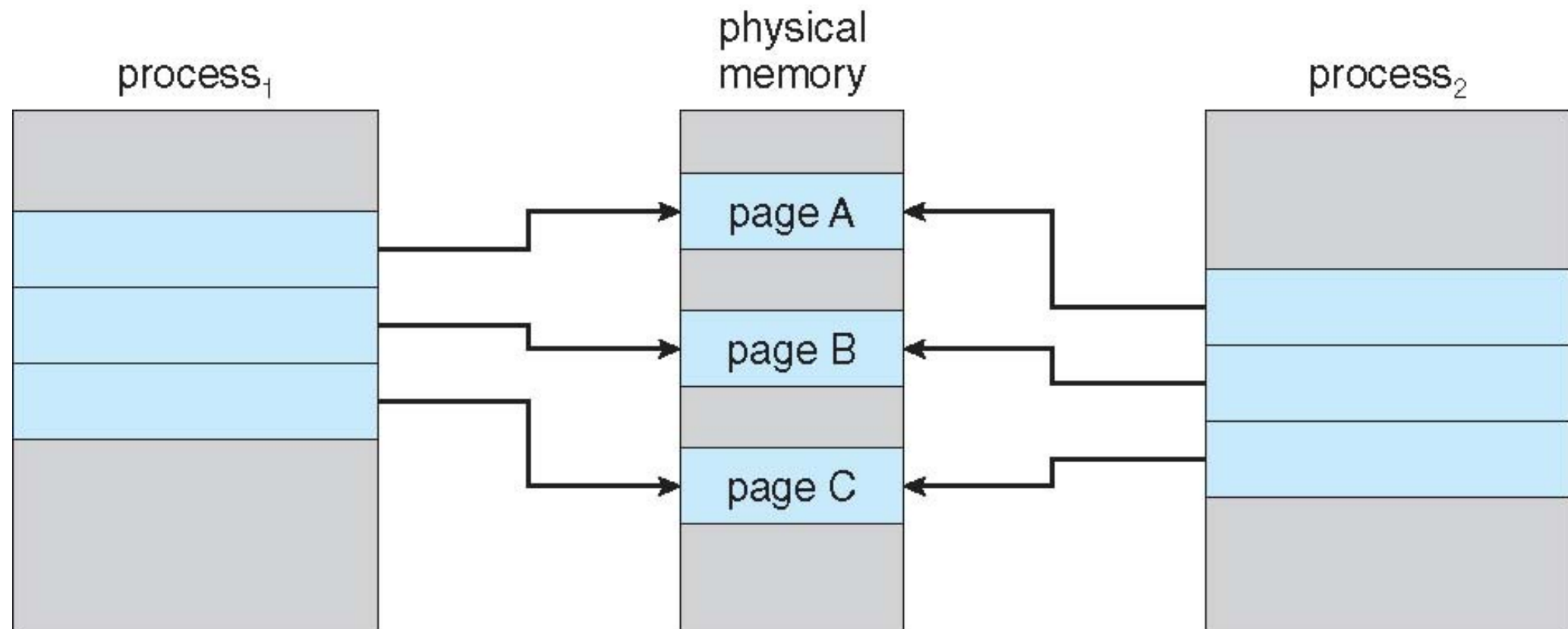
Agenda

- Recap / Introduction
- Demand Paging
- **Copy-on-Write**
- Page Replacement
- Allocation of Frames
- Thrashing
- Memory-mapped files [Self-reading]

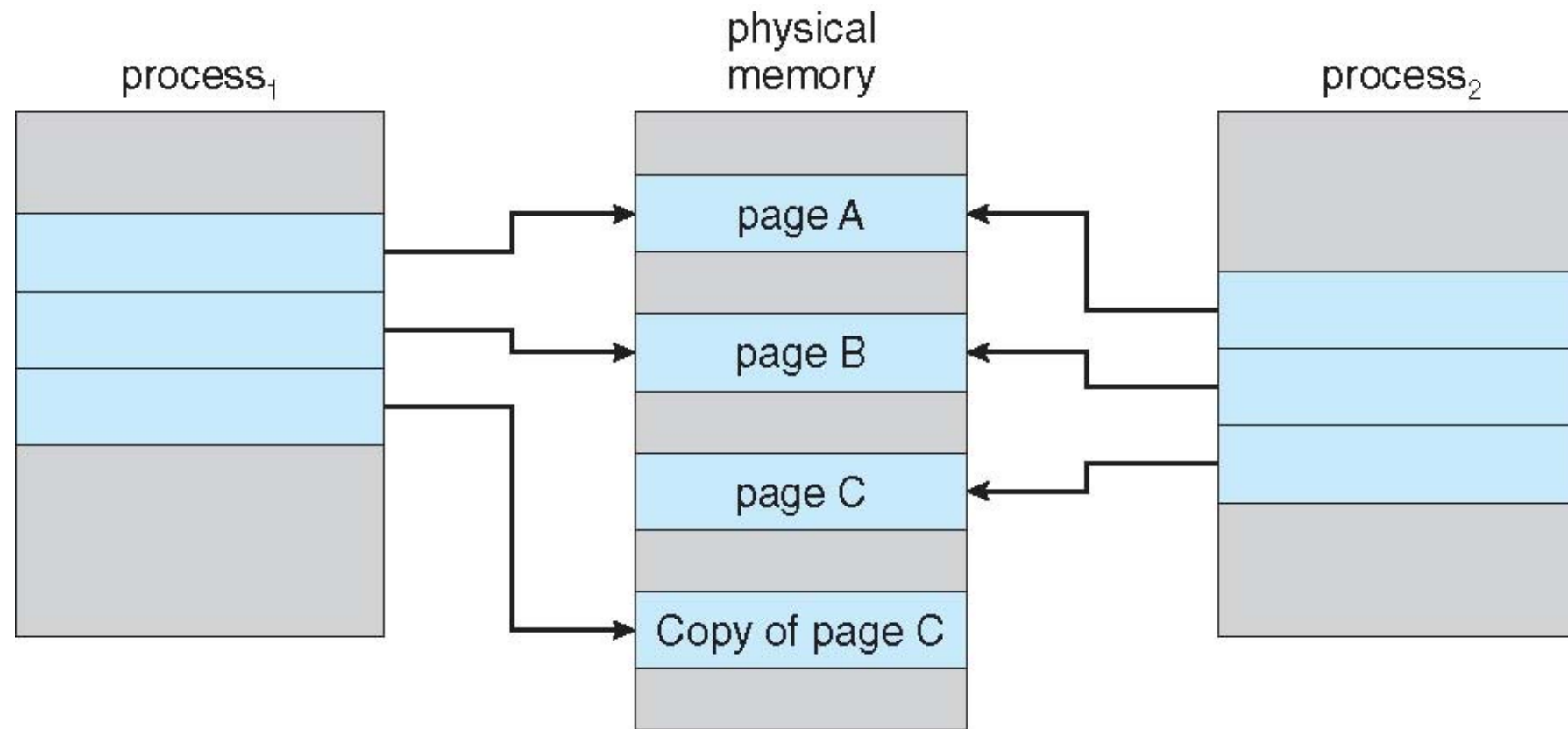
Copy-on-Write

- Example of optimization enabled by separation of Logical from Physical memory and by demand paging
- **Copy-on-Write** (COW) allows both parent and child processes to initially *share* the same pages in memory
 - If either process modifies a shared page, only then is the page copied
- COW allows more efficient process creation as only modified pages are copied
- `fork()` is commonly implemented using copy-on-write: parent and child initially share physical pages, and a page is copied when either process writes it.
- `vfork()` avoids creating a separate address space: the child temporarily runs in the parent's address space while the parent is suspended, so the child must promptly call `exec()` or `_exit()`.

Before Process 1 Modifies Page C



After Process 1 Modifies Page C





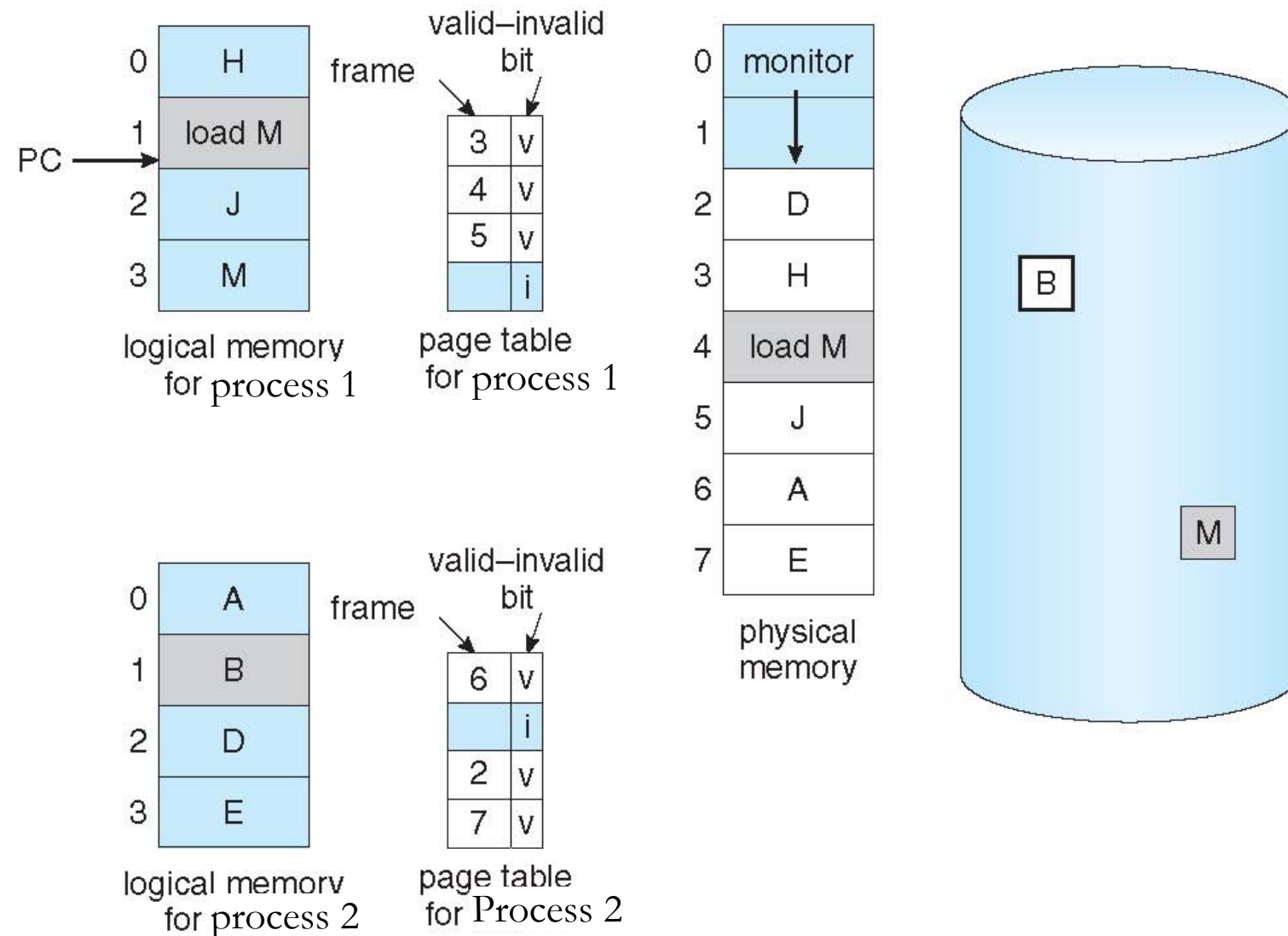
Dirty COW (CVE-2016-5195) was a Linux kernel race condition in copy-on-write handling that could allow an unprivileged process to modify read-only mappings and escalate privileges

Dirty COW (*Dirty copy-on-write*) is a **computer security vulnerability** for the **Linux kernel** that affects all Linux-based operating systems including **Android**. It is a local **privilege escalation** bug that exploits a **race condition** in the implementation of the **copy-on-write** mechanism.^{[1][2]} The bug has been lurking in the Linux kernel since version 2.6.22 (released in **September 2007**), and has been actively exploited at **least since October 2016**.^[2] The bug has been patched in Linux kernel versions 4.8.3, 4.7.9, 4.4.26 and newer.

Agenda

- Recap / Introduction
- Demand Paging
- Copy-on-Write
- **Page Replacement**
- Allocation of Frames
- Thrashing
- Memory-mapped files [Self-reading]

Suppose we end up in a situation like the following one



- *What to do now?*
- *Can we replace an existing page?*
- *Should it be from process 1 or can it also be from process 2?*
- *Is there any page that can be swapped out without writing it to disk?*
- *Is there any page that has lower probability of being needed later on?*
- *Could we have prevented this situation?*
- *How many pages should we have allocated to the OS and to processes 1 and 2 to prevent this?*
- ...

Process 1 needs a free frame (but 0 available!)

Page and Frame Replacement Algorithms



- *What to do now?*
- *Can we replace an existing page?*
- *Should it be from process 1 or can it also be from process 2?*
- *Is there any page that can be swapped out without writing it to disk?*
- *Is there any page that has lower probability of being needed later on?*
- *Could we have prevented this situation?*
- *How many pages should we have allocated to the OS and to processes 1 and 2 to prevent this?*
- ...

Frame-allocation algorithm

determines how many frames to give each process

Page-replacement algorithm

Which page to replace? Want lowest page-fault rate on both first access and re-access

Page Replacement

- When no free frame exists, the page-fault handler selects a victim page, evicts it (writing it first if dirty), and reuses its frame
- Use **modify** (**dirty**) **bit** to reduce overhead of page transfers – only modified pages are written to disk
- Page replacement completes separation between logical memory and physical memory – large virtual memory can be provided on a smaller physical memory

Basic Page Replacement

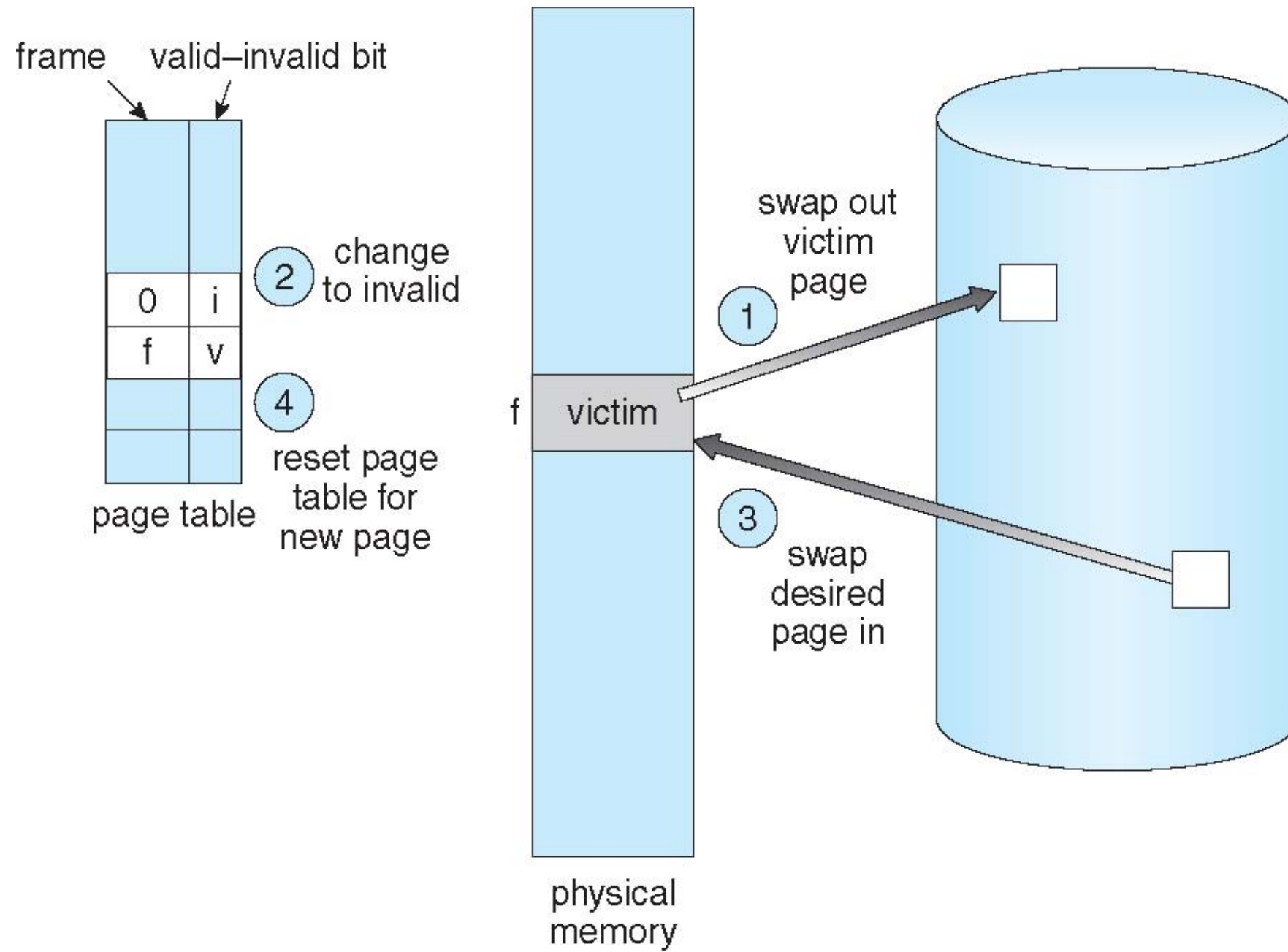
1. Find the location of the desired page on disk
2. Find a free frame:
 1. If there is a free frame, use it
 2. If there is no free frame, use a page replacement algorithm to select a **victim frame**
 1. Write victim frame to disk if dirty
3. Bring the desired page into the (newly) free frame; update the page and frame tables
4. Continue the process by restarting the instruction that caused the trap

Note now potentially 2 page transfers for page fault – increasing EAT

Page replacement algorithms

- We are going to look at:
- FIFO (First-In-First-Out)
- Optimal algorithm
- Least Recently Used (LRU)
- Approximated LRU

Page Replacement



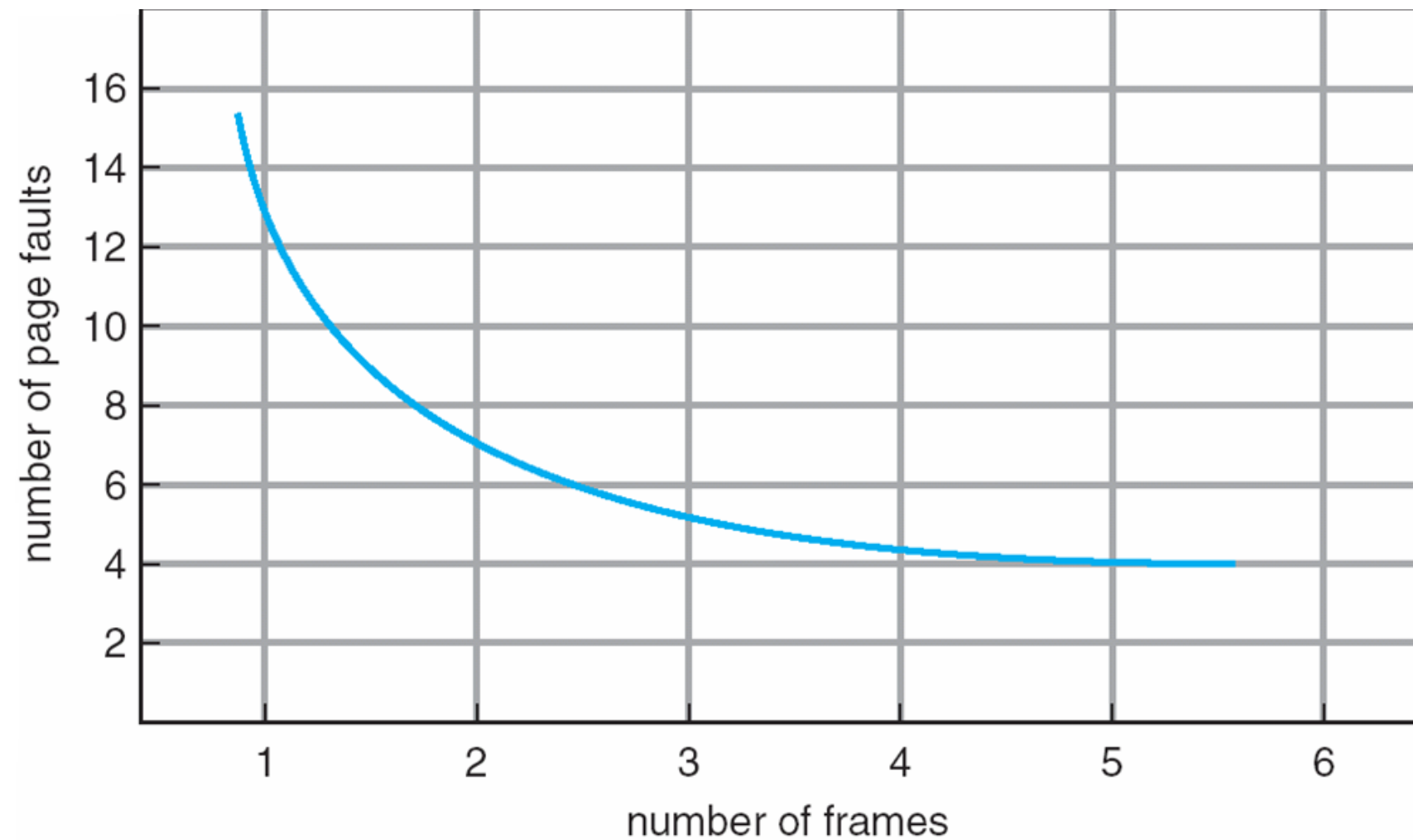
How to evaluate a page replacement algorithm?

- **Run** it on a particular string of memory references (reference string) and computing the number of page faults on that string
 - String is just page numbers, not full addresses
 - Repeated access to the same page does not cause a page fault (if the page is still resident)
 - Results depend on number of frames available
- In all our examples, the **reference string** of referenced page numbers is

7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1

What do we expect?

(Graph of Page Faults Versus The Number of Frames)



First-In-First-Out (FIFO) Algorithm

- If all frames are used, free the one with the page that has been resident the longest
- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)

reference string

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

Page needed: 7

Page faults: 0

Status of frames:



Is it there? NO

Do we have a free frame? YES

If YES, load the page

+1 page fault

First-In-First-Out (FIFO) Algorithm

- If all frames are used, free the one with the page that has been resident the longest
- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Page needed: 0

Page faults: 1

Status of frames:



Is it there? NO

Do we have a free frame? YES

If YES, load the page

+1 page fault

First-In-First-Out (FIFO) Algorithm

- If all frames are used, free the one with the page that has been resident the longest
- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Page needed: 1

Page faults: 2

Status of frames:

7	0	
---	---	--

Is it there? NO

Do we have a free frame? YES

If YES, load the page

+1 page fault

First-In-First-Out (FIFO) Algorithm

- If all frames are used, free the one with the page that has been resident the longest
- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Page needed: 2

Page faults: 3

Status of frames:

7	0	1
---	---	---

Is it there? NO

Do we have a free frame? NO

If NO, evict the page loaded more time ago (7)

+1 page fault

First-In-First-Out (FIFO) Algorithm

- If all frames are used, free the one with the page that has been resident the longest
- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Page needed: 0

Page faults: 4

Status of frames:

2	0	1
---	---	---

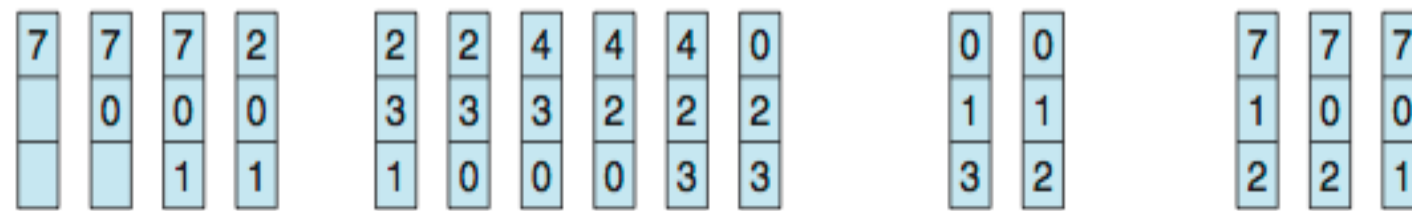
Is it there? YES

First-In-First-Out (FIFO) Algorithm

- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)

reference string

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

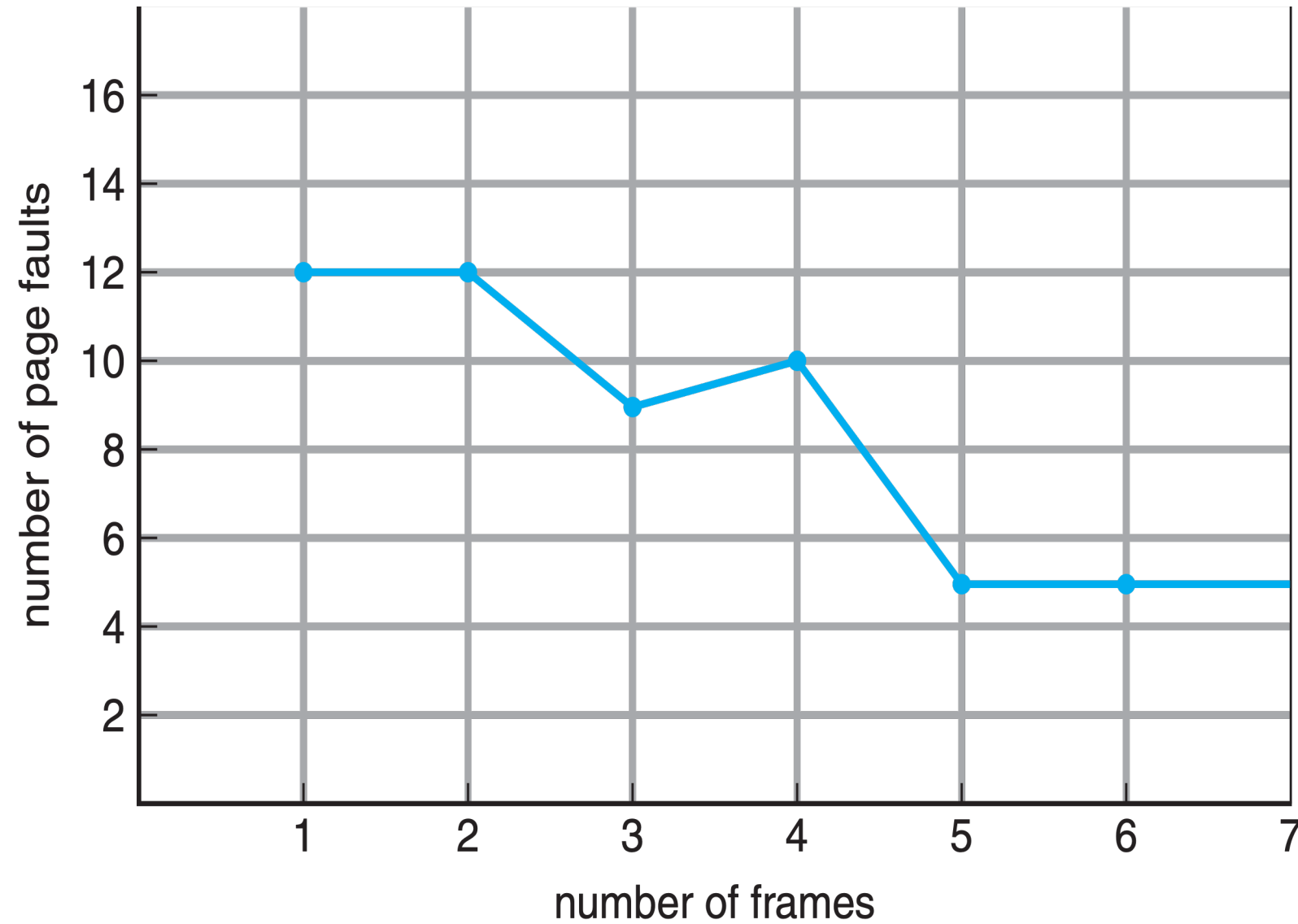


page frames

15 page faults

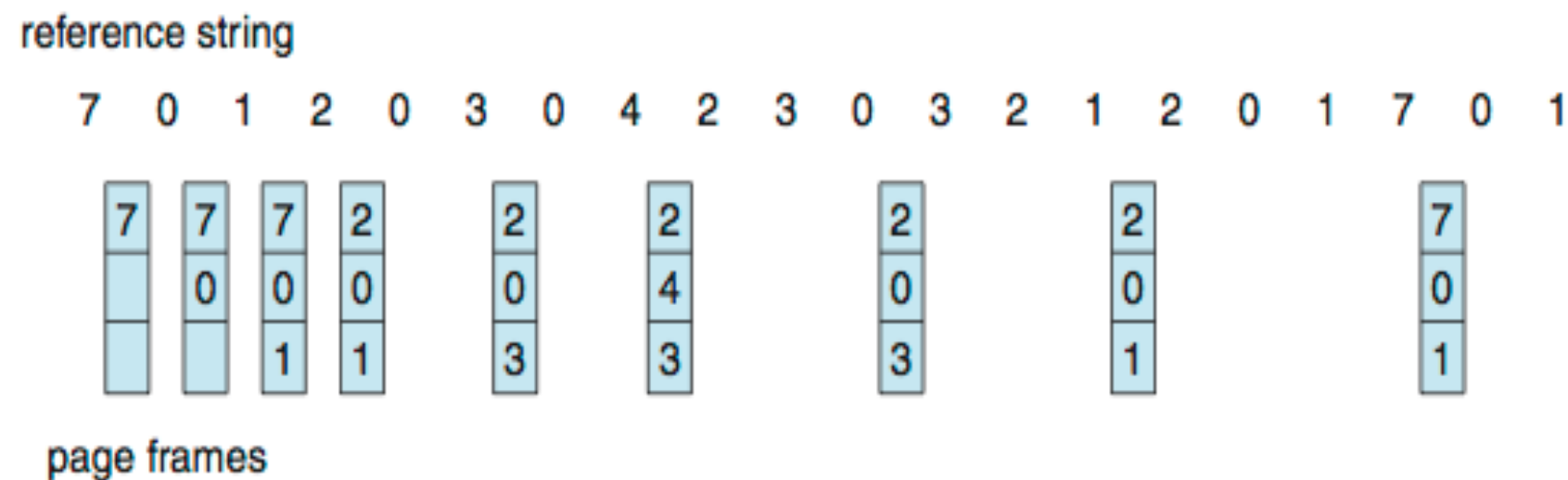
- Can vary by reference string: consider 1,2,3,4,1,2,5,1,2,3,4,5
- Adding more frames can cause more page faults!
 - **Belady's Anomaly**
- How to track arrival order of pages?
 - Just use a FIFO queue

FIFO Illustrating Belady's Anomaly



Optimal Algorithm

- Replace the page whose next use is farthest in the future
 - OPT produces 9 page faults for this reference string
- How do you know this?
 - Can't read the future
- OPT is useful as a benchmark but cannot be implemented exactly online because future references are unknown



Least Recently Used (LRU) Algorithm

- Use past knowledge rather than future
- Replace page that has not been used in the longest time
- Associate time of last use with each page

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0			1		1		1		
	0	0	0		0		0	0	3	3			3		0		0		
		1	1		3		3	2	2	2			2		2		7		

page frames

- 12 faults – better than FIFO but worse than OPT
- Generally good algorithm and frequently used (with approximations)
- But how to implement?

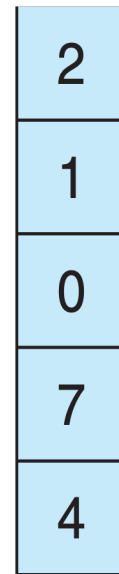
LRU Algorithm (Cont.)

- Counter implementation
 - Every page-table entry has a counter; every time page is referenced through this entry, copy the clock into the counter
 - When a page needs to be changed, look at the counters to find smallest value
 - Search through table needed
- Stack implementation
 - Keep a stack of page numbers in a doubly-linked list:
 - Page referenced → move it to the top
 - Each update more expensive
 - No search for replacement (page to replace at the bottom of the stack)

Use Of A Stack to Record Most Recent Page References

reference string

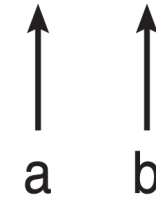
4 7 0 7 1 0 1 2 1 2 7 1 2



stack
before
a



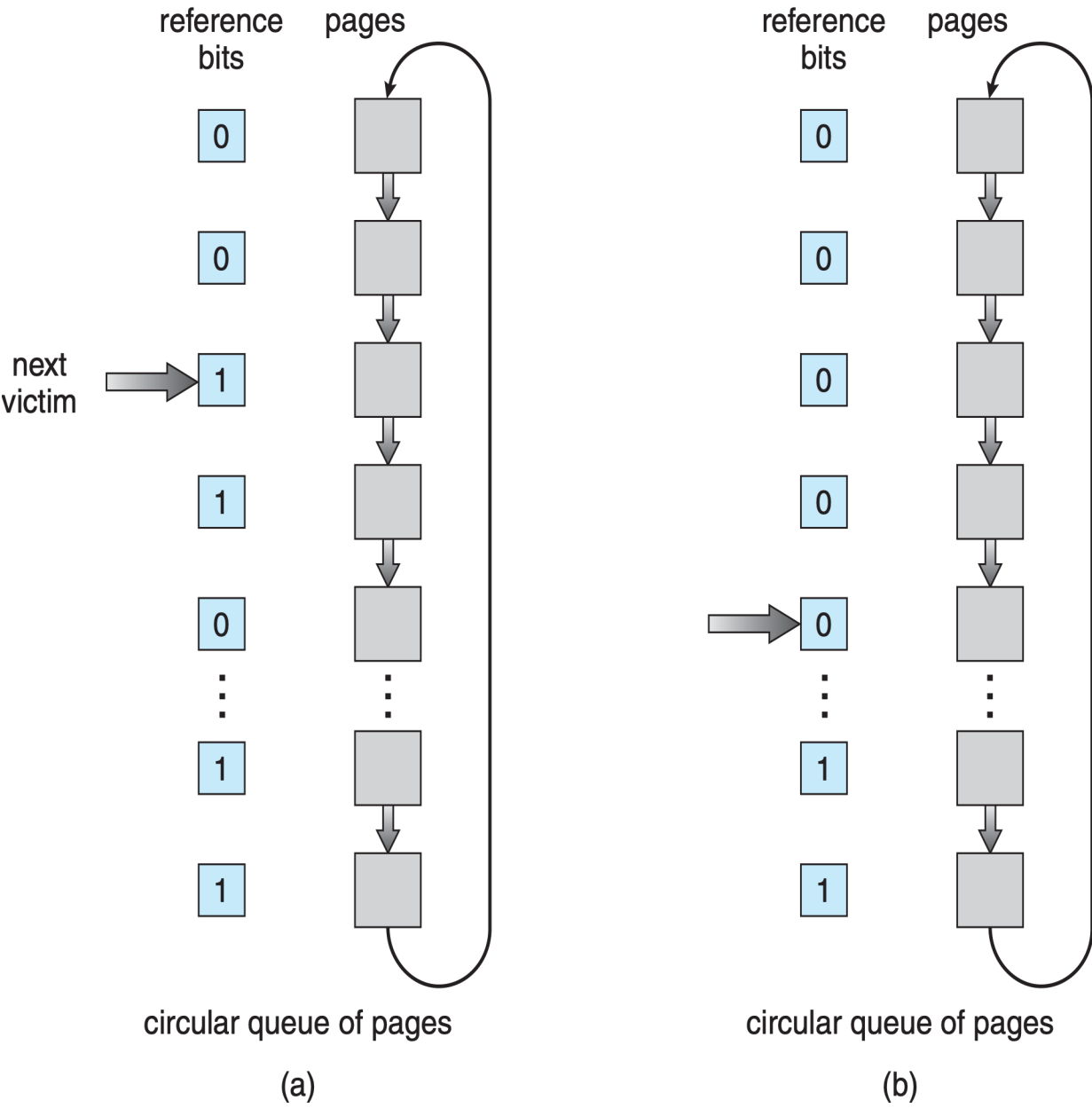
stack
after
b



LRU Approximation Algorithms

- LRU needs special hardware and still slow
- **Reference bit**
 - With each page associate a bit, initially = 0
 - Hardware sets a page's reference bit when the page is accessed; the OS periodically clears it
 - Replace any with reference bit = 0 (if one exists)
 - We do not know the order, however
- **Second-chance algorithm**
 - Generally FIFO, plus hardware-provided reference bit
 - **Clock** replacement
 - If page to be replaced has
 - Reference bit = 0 -> replace it
 - reference bit = 1 then:
 - set reference bit 0, leave page in memory
 - replace next page, subject to same rules

Second-Chance (clock) Page-Replacement Algorithm



Agenda

- Recap / Introduction
- Demand Paging
- Copy-on-Write
- Page Replacement
- Allocation of Frames
- Thrashing
- Memory-mapped files [Self-reading]

Page and Frame Replacement Algorithms



- *What to do now?*
- *Can we replace an existing page?*
- *Should it be from process 1 or can it also be from process 2?*
- *Is there any page that can be swapped out without writing it to disk?*
- *Is there any page that has lower probability of being needed later on?*
- *Could we have prevented this situation?*
- *How many pages should we have allocated to the OS and to processes 1 and 2 to prevent this?*
- ...

Frame-allocation algorithm

determines how many frames to give each process

Page-replacement algorithm

Which page to replace? Want lowest page-fault rate on both first access and re-access

Allocation of Frames

- Each process needs *minimum* number of frames
- *Maximum* of course is total frames in the system (but remember the OS and other processes need frames too)
- Two major allocation schemes
 - fixed allocation
 - priority allocation
- Many variations

Fixed Allocation

- Equal allocation – For example, if there are 100 frames (after allocating frames for the OS) and 5 processes, give each process 18 frames
 - Keep some as free frame buffer pool
- Proportional allocation – Allocate according to the size of process
 - Dynamic as degree of multiprogramming, process sizes change

– s_i = size of process p_i

– $S = \sum s_i$

– m = total number of frames

– a_i = allocation for $p_i = \frac{s_i}{S} \times m$

$$m = 64$$

$$s_1 = 10$$

$$s_2 = 127$$

$$a_1 = \frac{10}{137} \times 62 \approx 4$$

$$a_2 = \frac{127}{137} \times 62 \approx 57$$

Priority Allocation

- Use a proportional allocation scheme using priorities rather than size
- If process P_i generates a page fault,
 - select for replacement one of its frames (**Local replacement**)
 - select for replacement a frame from a process with lower priority process (**Global replacement**)

Global vs. Local Allocation

- **Global replacement** – process selects a replacement frame from the set of all frames; one process can take a frame from another
 - Global replacement can improve utilization or throughput, but it makes each process's performance less predictable. The preferred policy depends on the OS and workload.
- **Local replacement** – each process selects from only its own set of allocated frames
 - More consistent per-process performance
 - But possibly underutilized memory

Agenda

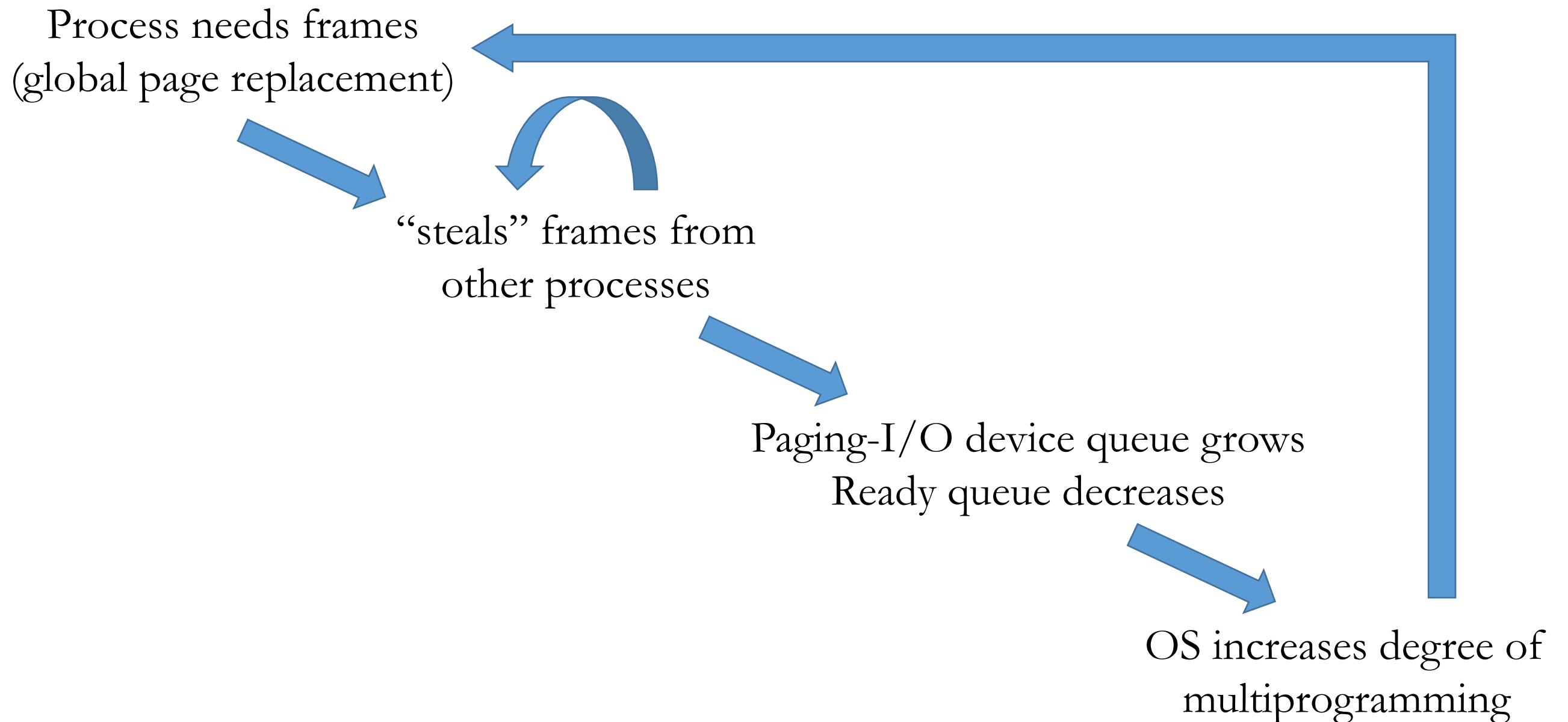
- Recap / Introduction
- Demand Paging
- Copy-on-Write
- Page Replacement
- Allocation of Frames
- Thrashing
- Memory-mapped files [Self-reading]



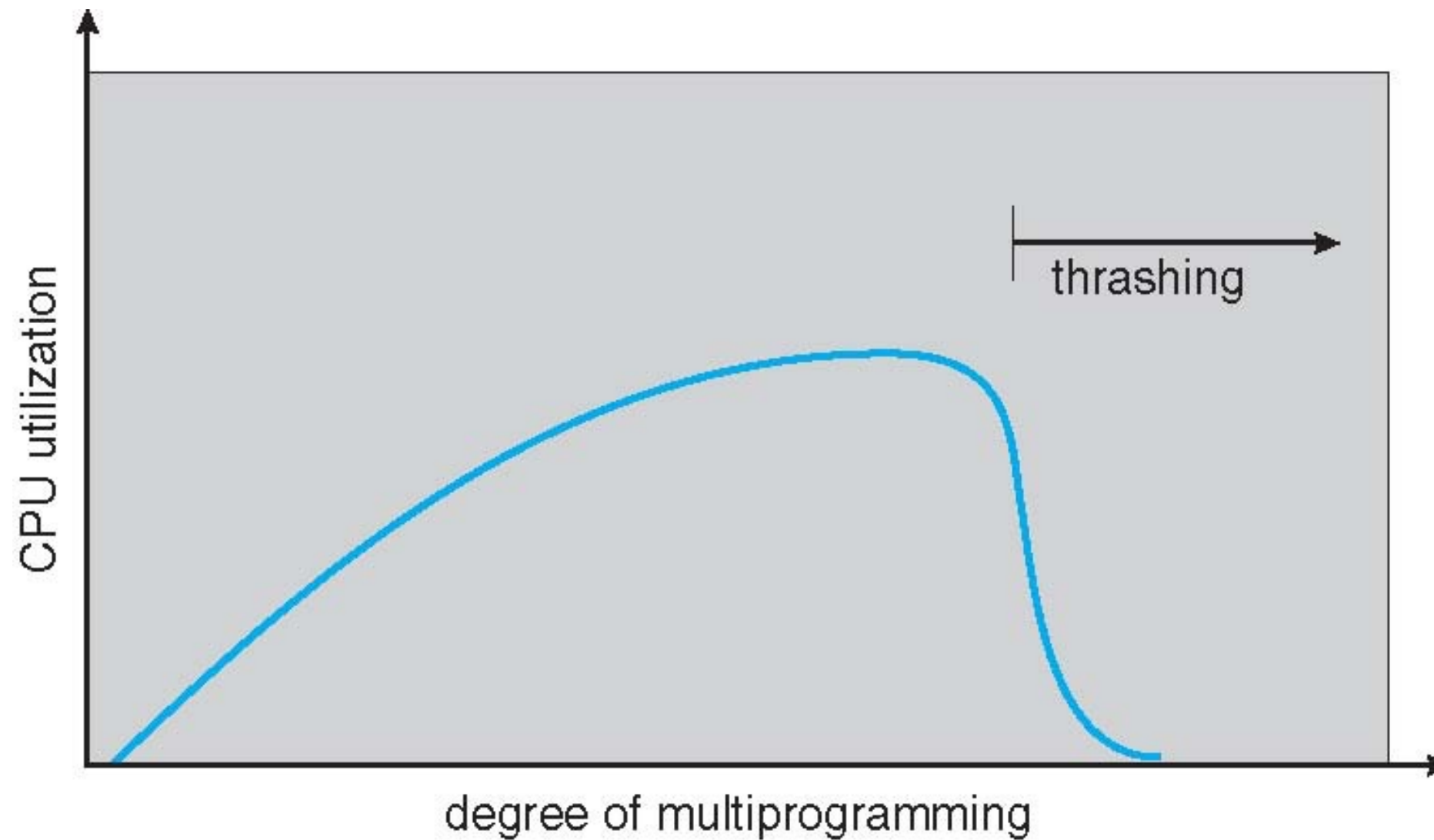
Thrashing

- If a process does not have “enough” pages, the page-fault rate is very high
 - Page fault to get page
 - Replace existing frame
 - But quickly need replaced frame back
 - This leads to:
 - Low CPU utilization
 - Operating system thinking that it needs to increase the degree of multiprogramming
 - Another process added to the system
- **Thrashing** $\equiv$ a process or system spends most of its time servicing page faults

Thrashing (cont.)



Thrashing (Cont.)



Sometimes the OS needs to reduce degree of multiprogramming to increase (restore) CPU utilization

Demand Paging and Thrashing

- Why does demand paging work?

Locality model

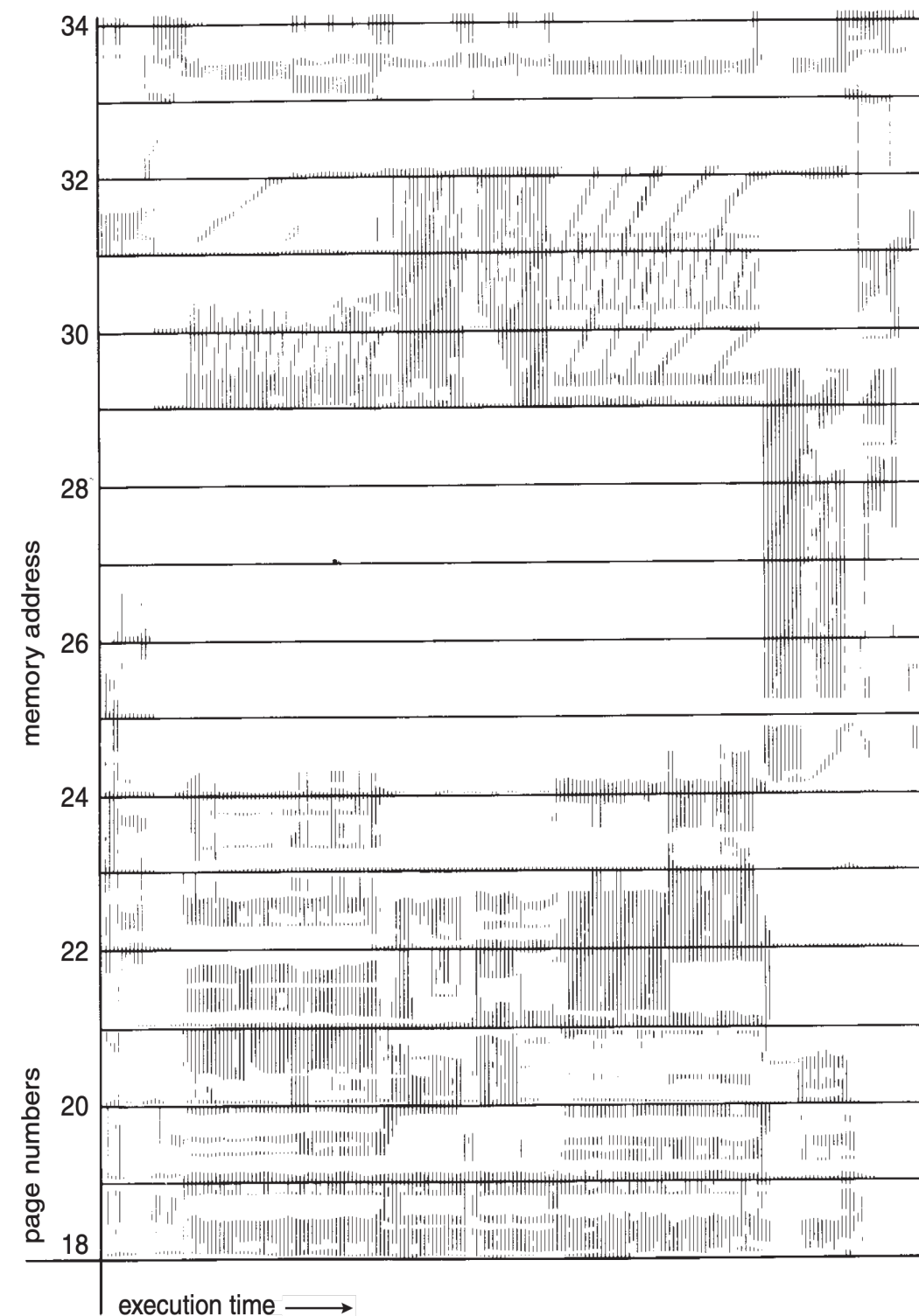
- Process migrates from one locality to another
- Localities may overlap

- Why does thrashing occur?

Σ size of locality > frames available to the active processes

- Limit effects by using local or priority page replacement

Locality In A Memory-Reference Pattern



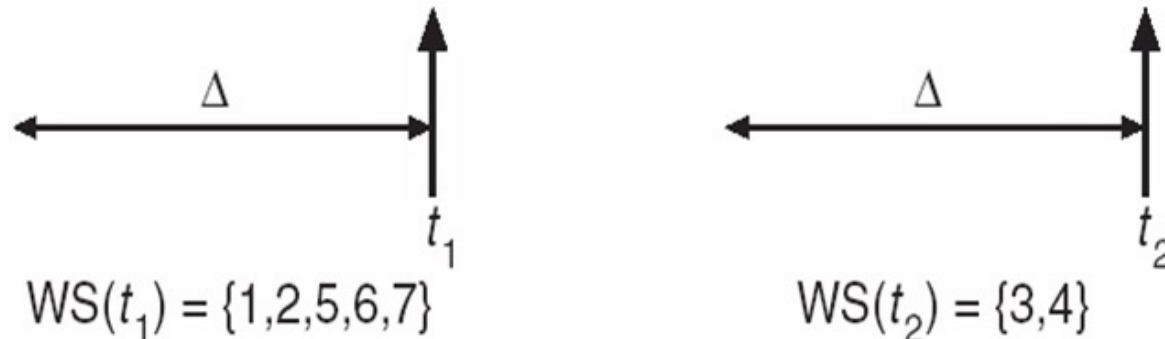
How could the OS avoid thrashing?



Working-Set Model

page reference table

... 2 6 1 5 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 3 4 3 4 4 4 1 3 2 3 4 4 4 3 4 4 4 ...

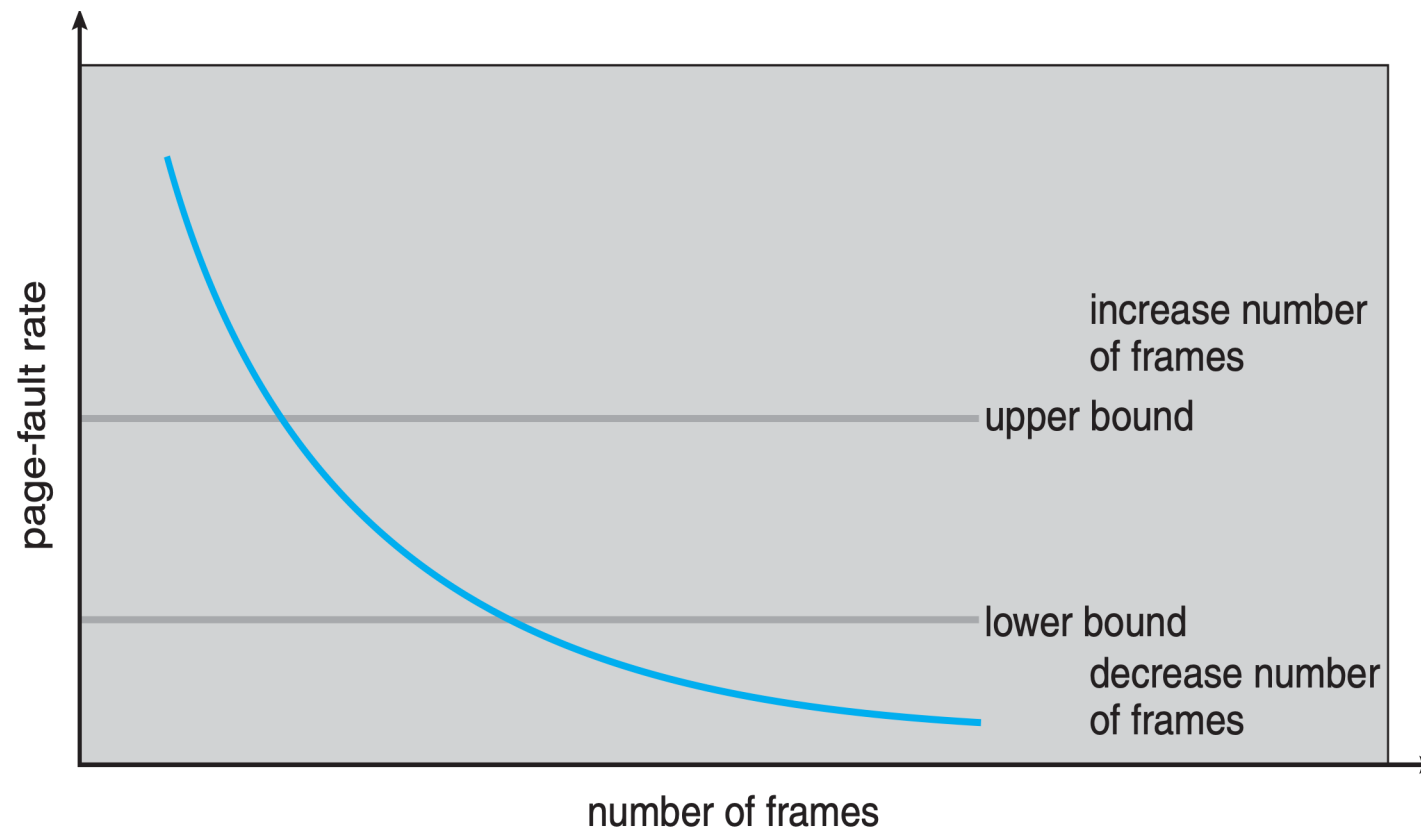


- $\Delta \equiv$ working-set window $\equiv$ a fixed number of page references
Example: 10,000 page references
- WSS_i (Working Set Size of Process i) =
total number of pages referenced by Process i in the most recent Δ (varies in time)
 - if Δ too small will not encompass entire locality
 - if Δ too large will encompass several localities
 - if $\Delta = \infty \Rightarrow$ will encompass entire program
- D (Total demand for frames) = $\sum WSS_i$
 - Approximation of locality
- if $D > \text{available frames } m \Rightarrow \text{Thrashing}$
- Policy: if $D > m$, then suspend or swap out one of the processes

How to keep track of the Working Set?

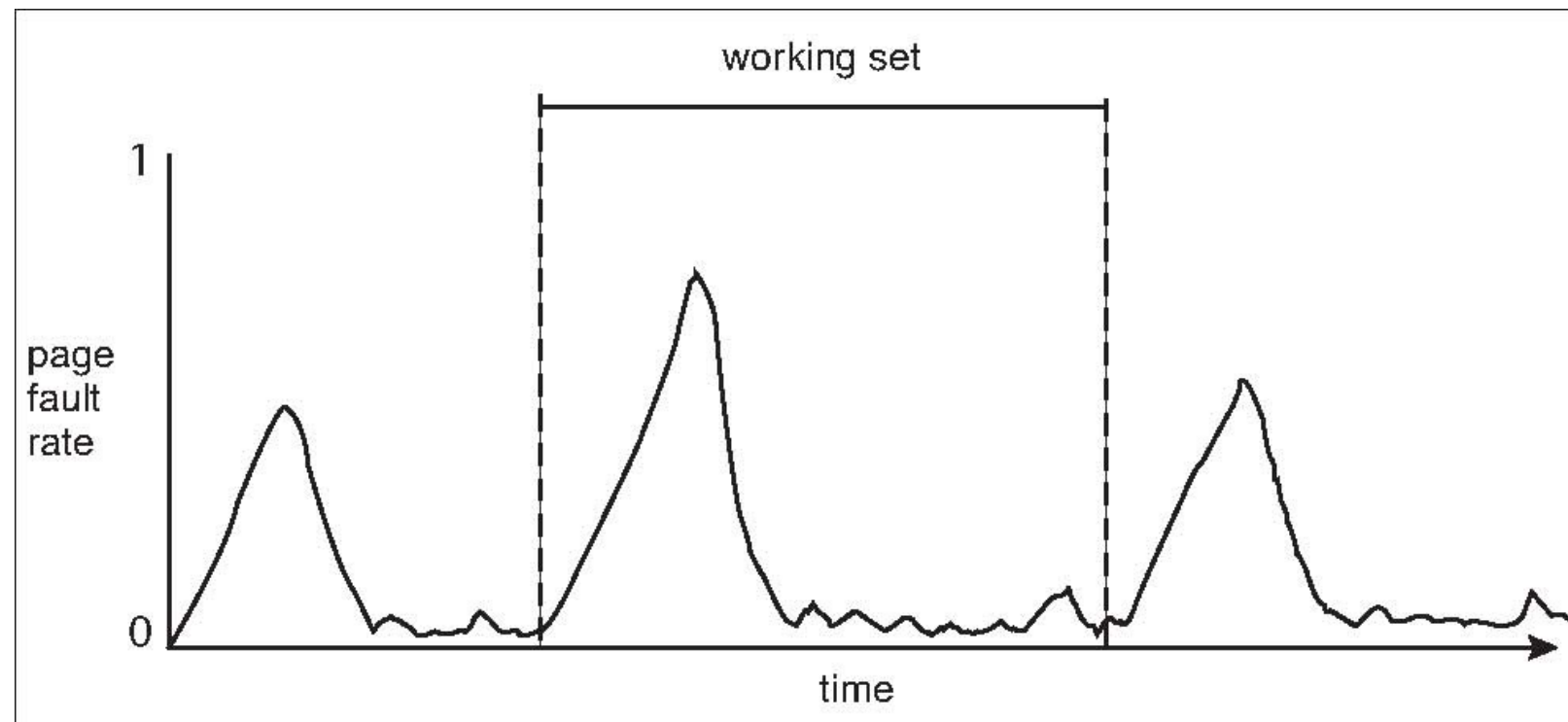
Page-Fault Frequency

- More direct approach than WSS
- Establish “acceptable” **page-fault frequency (PFF)** rate and use local replacement policy
 - If actual rate too low, process loses frame
 - If actual rate too high, process gains frame



Working Sets and Page Fault Rates

- The page-fault rate generally rises when a process has too few frames for its current working set and falls once enough frames are available.
- Working set changes over time
- Peaks and valleys over time



By looking at the Page Fault Rate you can observe the working set variations over time

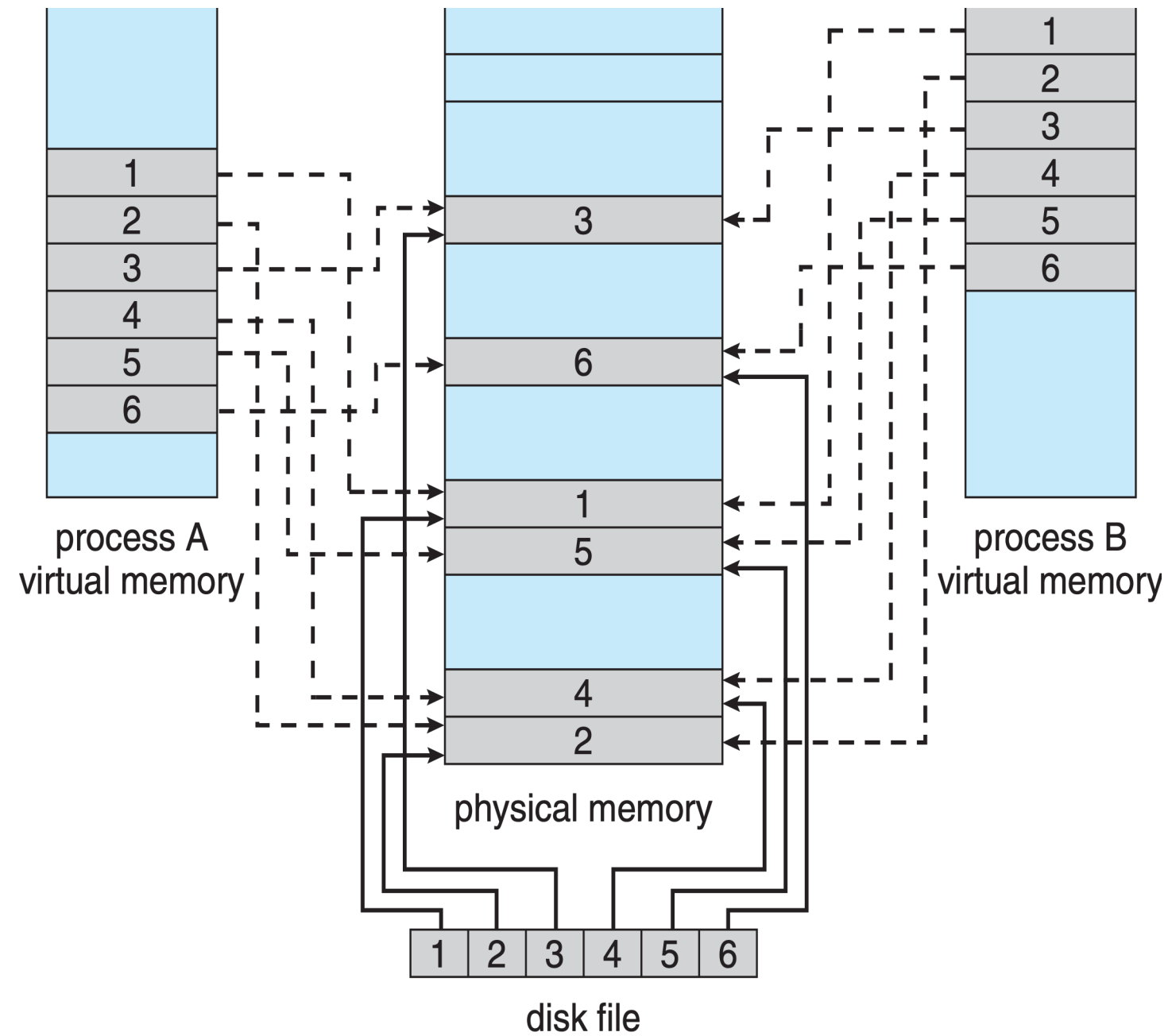
Agenda

- Recap / Introduction
- Demand Paging
- Copy-on-Write
- Page Replacement
- Allocation of Frames
- Thrashing
- Memory-mapped files [Self-reading]

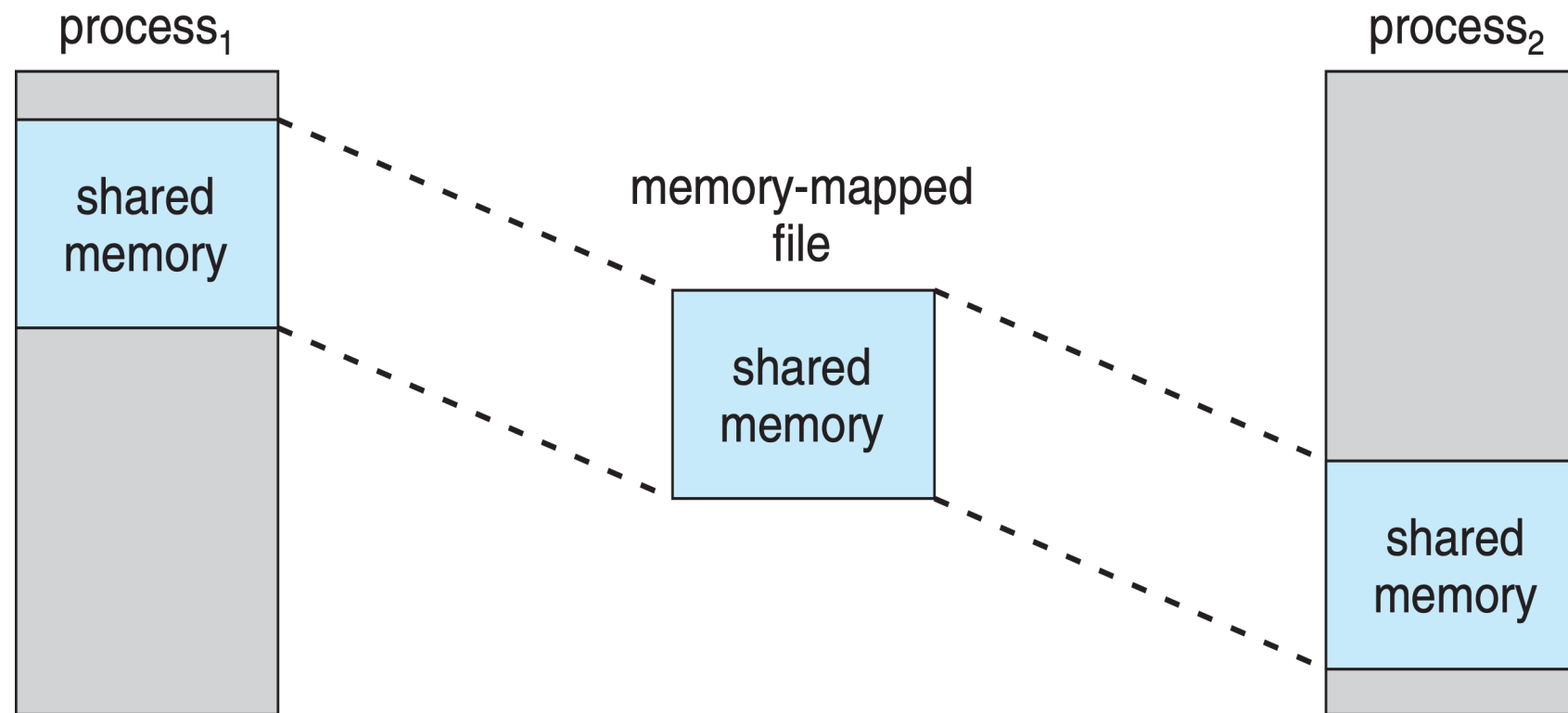
Memory-Mapped Files

- Memory-mapped file I/O allows file I/O to be treated as routine memory access by **mapping** a disk block to a page in memory
- A file is initially read using demand paging
 - A page-sized portion of the file is read from the file system into a physical page
 - Subsequent reads/writes to/from the file are treated as ordinary memory accesses
- Simplifies and speeds file access by driving file I/O through memory rather than `read()` and `write()` system calls
- Also allows several processes to map the same file allowing the pages in memory to be shared
- But when does written data make it to disk?
 - Periodically and / or at file `close()` time
 - For example, when the pager scans for dirty pages

Memory Mapped Files



Shared Memory via Memory-Mapped I/O



Some clarifications

- Are programs written to disk divided into pages? No (except swapped out pages)
- Can processes require extra pages? Yes, different portions of a page table can be loaded into the MMU during the execution of a process
- Are swapped pages written to disk? Yes, but sometimes dedicated portions of RAM can also be used (compressed pages)

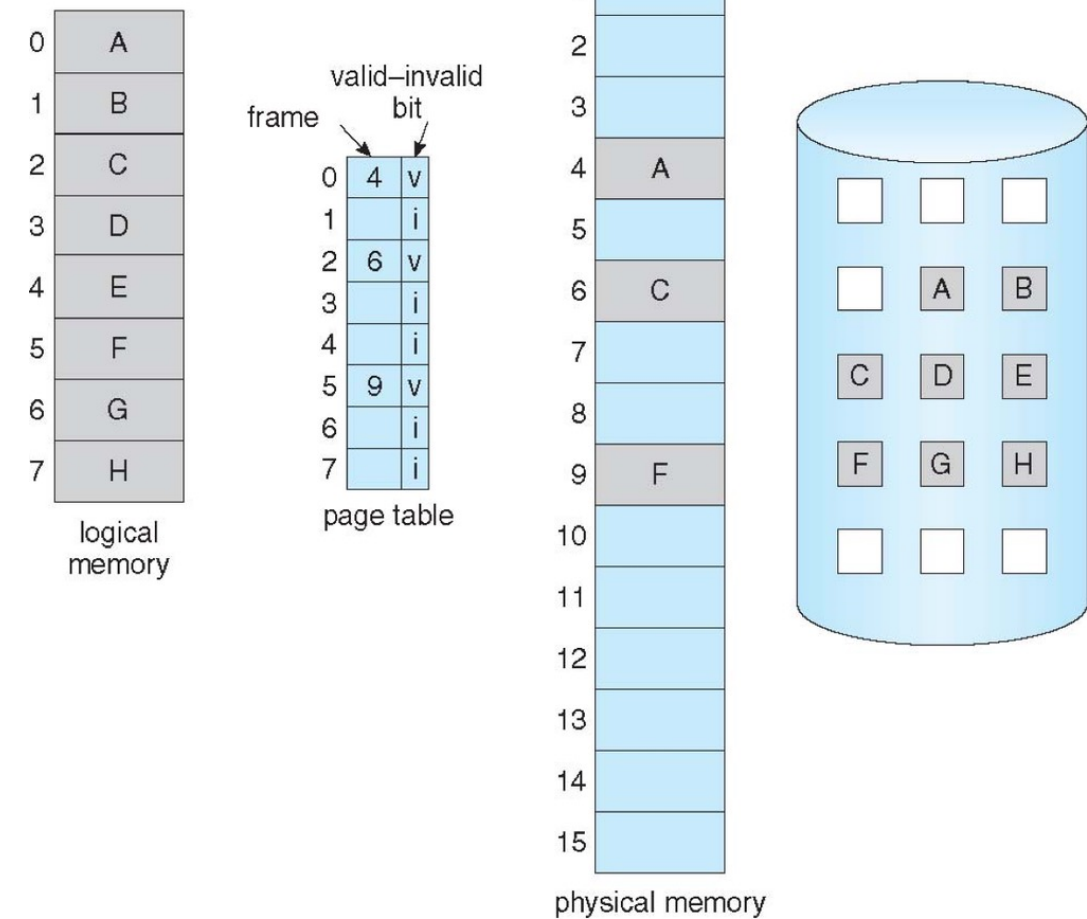
Questions

70

Vincenzo Gulisano

Access to page 3 generates a page fault

1. I agree
2. I don't think so...

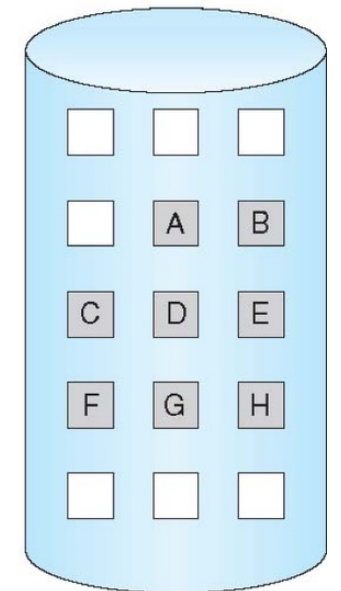
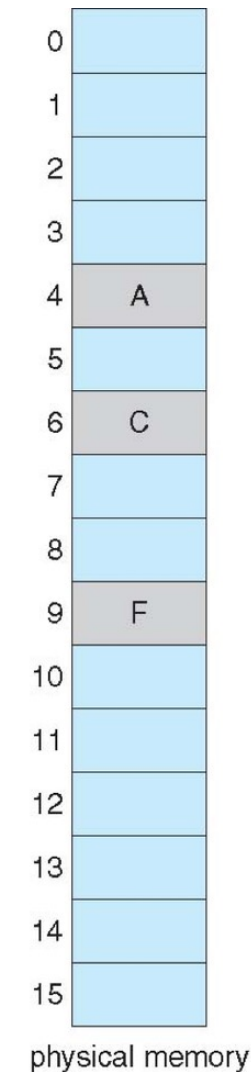
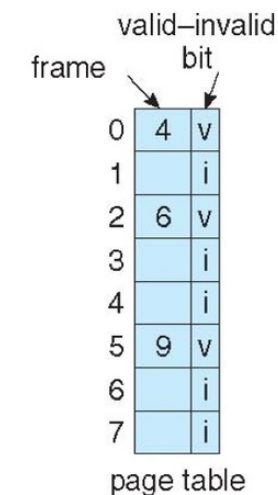


When using Virtual Memory, if there are 4 free frames and a process spans 5 pages...

1. The process cannot run
2. The process can run
3. Whether it can run depends on its concurrent execution
4. Whether it can run depends on its parallel execution

Access to pages 3,4,3 and 5 generates ...

1. 2 page faults
2. 4 page faults
3. at least 2 page faults
4. at least 4 page faults



If the invalid bit of a frame is set to 1, the page contained in the referenced frame does not belong to the process

1. Correct
2. Not correct

reducing the degree of multiprogramming can increase CPU utilization

1. Correct
2. Not correct

Having less available frames can reduce the number of page faults

1. Correct
2. Not correct

Exercise

Jake and Finn have a computer whose memory access time is 150 nanoseconds and whose page-fault service time is 5 milliseconds. A process previously run in an operating system without virtual memory was 2 times faster than the same process run by the actualized operating system (the latter having virtual memory). Every how many page accesses are Jake and Finn experiencing a page fault on average? Would buying a new disk able to bring the page-fault service time to 2 milliseconds make the process no more than 1.5 times slower in the new operating system (compared to the previous one)?

Exercise

- Given the following reference string

7 2 1 2 0 1 1 4 2 3 0 7 0 1 2 7 7 3 0 0 3 0 3

and assuming 4 frames are available, find:

- The minimum number of page faults we can observe
- The extra page faults caused by the FIFO replacement algorithm
- The extra page faults caused by the LRU replacement algorithm

Solution

OPTIMAL

7 2 1 2 0 1 1 4 2 3 0 7 0 1 2 7 7 3 0 0 3 0 3

7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	0	0	0	0	0
	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	1	4	4	3	3	3	3	3	3	3	3	3	3	3	3	3
				0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1

Page faults

1	1	1	0	1	0	0	1	0	1	0	0	0	1	0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

8

Solution

FIFO

7 2 1 2 0 1 1 4 2 3 0 7 0 1 2 7 7 3 0 0 3 0 3

7	7	7	7	7	7	7	4	4	4	4	4	4	4	2	2	2	2	2	2	2	2
	2	2	2	2	2	2	2	2	3	3	3	3	3	3	3	3	3	3	3	3	3
		1	1	1	1	1	1	1	1	1	7	7	7	7	7	7	7	0	0	0	0
				0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1

Page faults

1	1	1	0	1	0	0	1	0	1	0	1	0	1	1	0	0	1	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

11

Solution

LRU

7 2 1 2 0 1 1 4 2 3 0 7 0 1 2 7 7 3 0 0 3 0 3

7	7	7	7	7	7	7	4	4	4	4	7	7	7	7	7	7	7	7	7	7	7	7
	2	2	2	2	2	2	2	2	2	2	2	2	1	1	1	1	1	0	0	0	0	0
		1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	3	3	3	3	3
				0	0	0	0	0	3	3	3	3	3	2	2	2	2	2	2	2	2	2

Page faults

1	1	1	0	1	0	0	1	0	1	1	1	0	1	1	0	0	1	1	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

12

Exercise

- Given processes P_1, P_2, P_3 and P_4 having sizes of 20, 25, 50 and 4 pages and given a total number of frames equal to 80 (10% of which reserved to the OS), compute the number of frames allocated to each process if frames are allocated proportionally to the size of each program

Exercise

- Suppose a process P has size of 100 bytes. Compute (1) the number of wasted bytes caused by internal fragmentation if the page and frame sizes are set to 2^4 bytes and (2) the size in bits of the page table (assume each frame entry requires 1 byte and that dirty bits are also used to speed the swap out of pages).